An Empirical Study of Adopting UDF Outlining in Modern Databases

Submitted in partial fulfillment of the requirements for

the degree of

Master of Science

in

Information Networking

Yuchen Liu

B.Eng., Compute Science, University of Hong Kong

PREVIEW

Carnegie Mellon University
Pittsburgh, PA

December, 2024

PREVIEW

© Yuchen Liu, 2024
All Rights Reserved

Acknowledgements

I would like to express my heartfelt gratitude to my advisors, Prof. Andy Pavlo and Prof. Todd Mowry, for their unwavering support and guidance throughout this research. Prof. Pavlo, your unparalleled expertise in databases and your drive for innovation have been instrumental in shaping the direction of this work. Prof. Mowry, your profound understanding of compilers and your ability to illuminate complex concepts have significantly enhanced my approach to tackling challenges in this thesis.

I am deeply indebted to Sam Arch, whose dedication and insight were pivotal in the success of this project. Working closely with Sam, I benefited from not only his technical acumen but also his collaborative spirit and mentorship, which made this journey both enriching and enjoyable.

I also want to thank my peers and colleagues at Carnegie Mellon University for fostering a vibrant and supportive academic environment. Their feedback, discussions, and encouragement have played an essential role in the development of this thesis.

Finally, I extend my deepest appreciation to my family and friends for their unwavering support and belief in me throughout this journey.

Thank you all for your contributions to this milestone.

This project was self funded by the author.

PREVIEW

Abstract

User-Defined Functions (UDFs) play an integral role in enhancing database extensibility and supporting complex queries, yet their usage often imposes performance degradation due to context-switching overheads and optimization boundaries. UDF outlining is the state-of-the-art optimization technique that inherits UDF inlining's ability to eliminate such constraints while avoiding its drawback of generating complex subqueries that leads to slow execution plans from the database optimizer. This thesis thoroughly examines the design decisions and tradeoffs of PRISM, a UDF optimization module supporting UDF outlining. Through a combination of experimental benchmarking and performance profiling, this study evaluates the impact of adopting PRISM in four modern database systems. The results highlight the applicability of PRISM and the conditions under which it yields substantial performance improvements. Insights gained from this analysis demonstrate the effectiveness of enhancing the execution efficiency of UDFs through the introduction of dedicated PRISM-like UDF optimization modules to database systems.

PREVIEW

Table of Contents

Acknowledgements **ii**

Abstract **iii**

List of Tables **vii**

List of Figures **viii**

List of Listings **ix**

1 Introduction **1**

1.1 User-Defined Functions 1

1.2 UDF Optimizations 3

1.3 Contributions 3

1.4 Outline 4

2 Background **5**

2.1 UDF Compilation and Query Compilation 5

2.2 UDF Inlining 6

2.3 Subquery Decorrelation 7

2.4 UDF Outlining 8

3 Design of PRISM **9**

3.1 Parser 11

3.1.1 Static-Single-Assignment Style Control Flow Graph 11

3.1.2 Implementation 11

3.2 Optimizer 12

PREVIEW

3.2.1 Data Flow Analysis 13
3.2.2 Query Motion 14
3.2.3 Instruction Elimination 15
3.2.4 Region-Based UDF Outlining 16
3.2.5 Predicate Hoisting 18
3.2.6 Aggify 19
3.2.7 Other Optimizations 20
3.2.8 Pass Management 20
3.3 UDF Outliner 21
3.3.1 Compilation vs. Interpretation 22
3.3.2 Compilation-Based Outlining for Interpreting Engines 22
3.3.3 Interpretation-Based Outlining for Interpreting Engines 27
3.3.4 Compilation-Based Outlining for Compiling Engines 27
3.4 UDF Inliner 28
3.4.1 CFG to UDF Conversion 28
3.4.2 UDF Registration 30
3.4.3 Improvements to Apfel 30
3.4.4 Will PRISM Work without Apfel? 31
3.5 Catalog Support 32
3.5.1 Binding of UDF Local Variables 32
3.5.2 Inlined UDF Registration 33
3.5.3 Outlined UDF Registration 34

4 Execution Model **35**
4.1 At UDF Creation Time 35
4.2 At Query Runtime 36

5 Case Studies **37**
5.1 DuckDB 38
5.1.1 UDF Compilation in DuckDB 38
5.1.2 Experiment Evaluation 41
5.2 SQL Server 43

PREVIEW

5.2.1 Integration of PRISM 43
5.2.2 Experiment Evaluation 43
5.2.3 Performance Bottlenecks 45
5.3 PostgreSQL 45
5.3.1 Integration of PRISM 46
5.3.2 Performance Estimation 46
5.4 Umbra 47
5.4.1 Integration of PRISM 47
5.4.2 Performance Estimation 48
5.5 Takeaways 48

6 Future Works **50**

7 Conclusion **52**

Bibliography **53**

Appendix A **58**

PREVIEW

List of Tables

Table 5.1 Speedup Statistics of PRISM Compared to UDF Inlining in DuckDB 41
Table 5.2 Top 3 Factors Contributing to the Speedup of PRISM in DuckDB 42
Table 5.3 Speedup Statistics of PRISM Compared to UDF Inlining in SQL Server 43
Table 5.4 Top 3 Factors Contributing to the Speedup of PRISM in SQL Server 44

PREVIEW

List of Figures

Figure 3.1 The system architecture of PRISM. 10
Figure 3.2 The internal workflow of the Parser. 12
Figure 3.3 Basic idea of Query Motion. 14
Figure 3.4 (a) Query Motion should stop moving queries forward to avoid further fragmentation. (b) Query Motion should also move procedural codes that block the query with the query. 16
Figure 3.5 Pipeline of the passes in the Optimizer of PRISM. 21
Figure 3.6 Example CFG for a conditional branch. 24
Figure 3.7 Steps to compile a SQL expression. (a) is a table of LLVM bitcode for each operator. (b) is the execution plan of a SQL expression. (c) is the LLVM IR generated from the execution plan. 26
Figure 3.8 The internal workflow of the UDF Inliner. 28

PREVIEW

Listings

1.1 An UDF example written in PL/pgSQL. 2

2.1 An UDF example written in PL/pgSQL and the calling query. . 7

3.1 An example of a UDF that can be outlined. 17
3.2 The rewritten version of the UDF in Listing 3.1. 17
3.3 The UDF of Listing 3.2 after outlining. 18
3.4 The C++ code generated from the control flow in Figure 3.6. . 25
3.5 The C++ code generated from three types of instructions: assignment, return, and exception. 25
3.6 An example UDF snippet that can be efficiently compiled by Froid. 31
3.7 An example of a UDF that uses local variables. 32
3.8 SQL command to create a temporary table for the UDF in Listing 3.7. 33

5.1 C++ code to register a custom operator in DuckDB extension. 39
5.2 The vectorized function implementation of **addAbs** in DuckDB. 40

A.1 The outlined body of the `addAbs` UDF. 59

PREVIEW

1

Introduction

1.1 User-Defined Functions

Databases were initially designed to store and retrieve data. However, as the complexity of data processing increased, the need for more sophisticated and custom operations arose. In the 1980s, Oracle RDBMS incorporated stored procedures that encapsulates a series of SQL statements into a single unit of work [49]. However, stored procedures are not flexible enough to embed into analytical queries. User Defined Functions (UDFs) were introduced as part of the broader support for programmability within Database Management Systems(DBMSs). They allow users to create functions to perform logic that are not available using built-in Structured Query Language (SQL) operations. The early 1990s saw commercial databases like Sybase, Oracle, and SQL Server adopt the idea of reusable, user-defined subroutines [49]. Nowadays, most mainstream databases support UDFs, which can be expressed in various programming languages, including PL/SQL, Python, Java, and C++.

One of the most popular ways to write UDFs is by some kind of imperative SQL language, referred to as PL/SQL. Like SQL has its standard [15], SQL/PSM

PREVIEW

```sql
-- UDF:
CREATE FUNCTION splitBill(headcount int, billId int) RETURNS DECIMAL AS
$$
DECLARE
    split DECIMAL = 0;
BEGIN
    IF headcount > 0 THEN
        split = (SELECT amount / headcount FROM payment WHERE id = billId);
    END IF;
    RETURN split;
END
$$ LANGUAGE plpgsql;

-- Query:
SELECT billdId, splitBill(headcount, billId) from events where eventId = 4;
```

Listing 1.1: An UDF example written in PL/pgSQL.

defines the standard of PL/SQL [48], but databases usually develop custom dialects of the language. For example, PostgreSQL comes with PL/pgSQL, Microsoft SQL Server uses Transact-SQL [33], and Oracle RDBMS introduces PL/SQL [6]. While each of them has different syntactic features, all of them share the same rationale of linking declarative, relational SQL with imperative constructs. For example, Listing 1.1 shows a pair of PL/SQL UDF and SQL query. The UDF written in PL/pgSQL calculates the splitted bill for a group of people, and the query displays all the splitted bills of event 4. After a PL/SQL UDF is created, it can be called in a SQL query like other built-in functions.

The query optimizer in DBMSs optimizes declarative SQL queries by picking an execution path that has the lowest estimated cost. However, it does not understand the imperative constructs in PL/SQL UDFs, so UDFs are ignored during query optimization. The mismatch of programming paradigm between SQL and UDF has led to inadequate performance of PL/SQL UDF since its debut. Before UDF compilation came out in 2001 [38], databases at the early days execute UDFs by interpreting them row by row, known as RBAR execution.

PREVIEW

1.2 UDF Optimizations

Researchers proposed various optimization techniques for PL/SQL UDFs, including UDF compilation and UDF inlining. UDF compilation compiles the UDF into native machine code to eliminate the interpretation overhead [27, 38]. UDF inlining translates the UDF to an equivalent SQL statement and inlines it to the calling query [42, 31]. While UDF compilation can sometimes decrease UDF execution time by 10× [27], UDF inlining showed higher potential because it targets the fundamental issue of paradigm disconnect. One problem with the inlining approach though is that it often generates huge subqueries that are difficult for the query optimizer to decorrelate, and considerable lateral joins and recursive CTEs (Common Table Expression) that are slow to execute [17].

To improve on UDF inlining, we proposed a new approach called UDF outlining that can greatly reduce the size of the subquery that will be inlined [17]. The idea is to deconstruct the UDF into procedural codes and relational SQL queries. The procedural parts get outlined away and hidden from the query optimizer to avoid confusion, whereas the relational parts are inlined to the calling query to let the query optimizer pick a better execution plan. We implemented this approach by creating a UDF optimization framework named PRISM. Experiments show that the outlining approach can lead to a geometric average speedup of 4.1× on UDF execution in DuckDB and 17× in Microsoft SQL Server compared to the pure inlining approach.

1.3 Contributions

The paper on UDF outlining introduces the primary algorithms of method. However, it provides limited details regarding the implementation of the technique. For developers seeking to integrate this method into in their database systems, additional implementation insights are necessary. This thesis aims to act as a comprehensive

PREVIEW

reference for subsequent discussion on the outlining approach. Specifically, the contribution of this thesis are as follows:

1. Describe the design options for each component of PRISM and their tradeoffs.
2. Call attention to future research directions related to each component and optimization pass of PRISM.
3. Discuss how several modern database systems can adopt the UDF outlining technique, the performance benefit and bottleneck, and how the findings apply to other databases.
4. Emphasize the importance of having a dediciated UDF optimization module like PRISM in state-of-the-art databases.

1.4 Outline

The thesis is composed using the following structure. Chapter 2 will first introduce related work to UDF outlining and then illustrate the UDF outlining technique by walking through the five components of the PRISM framework. Chapter 3 will describe the options to implement each component and their tradeoffs. Chapter 4 will summarize the functionality of each component and use an example UDF to illustrate the execution model within PRISM. Chapter 5 will discuss potential paths to implement outlining in four modern databases, namely DuckDB, SQL Server, PostgreSQL and Umbra. For each of the DBMS, we will evaluate the potential performance gain and its implication. Finally, Chapter 6 will enumerate future works related to UDF outlining and PRISM.

PREVIEW

2

Background

This chapter introduces two popular UDF optimization techniques that came before UDF outlining: UDF compilation and UDF inlining. Both approaches inspired the development of UDF outlining, and became indispensable building blocks for it.

2.1 UDF Compilation and Query Compilation

UDF compilation is the technique that compiles UDF to native machine code. It helps to eliminate the interpretation overhead from RBAR execution. Compilation support for PL/SQL UDFs in Oracle RDBMS dates back to 2001 [38]. It first converts PL/SQL Stored Procedures to Pro*C [47] and then compiles it to normal C code. The C code can be compiled into a shared library and linked to the database runtime instance. Microsoft SQL Server added compilation for its T-SQL UDFs in 2016 [33]. Both of the systems cannot compile UDFs with SQL queries.

UDF compilation represents a specialized usage of query compilation. Query compilation is the execution strategy proposed by Chamberlin et al. [23] that first compiles a SQL query to machine code or byte code, then executes the generated code [23, 28]. Compared with traditional volcano-style query execution engines,

PREVIEW

Reproduced with permission of copyright owner. Further reproduction prohibited without permission.